

Tập 5 - Giải phẫu CNI

Đặc tả CNI Specification & Tự tay cắm mạng thủ công bằng `cnitool`

Phần 0 — Nền tảng K8s Networking · `#CNI` `#CNISpec` `#cnitool` `#linux-net`



CNI

Lộ trình bài học: Lab First - Slide Second

Hôm nay chúng ta sẽ mở toang chiếc "hộp đen" mang tên CNI:

- **PHẦN 1: Thực hành đóng vai (Lab First)**

- Đột nhập `/opt/cni/bin` để truy tìm các file binary vô tri.
- Soạn thảo hợp đồng mạng JSON `.conflist` chuẩn đặc tả.
- Tạo Network Namespace Linux thủ công.
- Đóng vai làm Kubelet dùng `cnitool` gọi CNI cắm/rút mạng (`ADD` / `DEL`).

- **PHẦN 2: Giải phẫu đặc tả (Slide Second)**

- Phân tích bản chất chuẩn đặc tả CNI Specification.
- Giải nghĩa các biến môi trường và JSON đầu vào/đầu ra.
- Kiến trúc tương tác giữa Kubelet và các CNI Plugins trong cụm.

PHẦN 1: Bắt đầu làm Lab ngay!

Chúng ta sẽ thực hiện các Thí nghiệm đột phá trong file `lab-guide.md`:

1. **Thí nghiệm 1:** SSH vào `worker1`. Liệt kê các file binary CNI vô tri nằm trong `/opt/cni/bin/` (ví dụ: `bridge`, `host-local`, `loopback`).
2. **Thí nghiệm 2:** Tự tay viết một hợp đồng cấu hình mạng JSON `mynet` có địa chỉ IPAM tĩnh `10.99.0.0/24`.
3. **Thí nghiệm 3:** Khởi tạo Network Namespace `cni-test` và dùng `cnitool add` để ra lệnh cho CNI cắm mạng. Chứng minh namespace có card `eth0` và IP thành công!
4. **Thí nghiệm 4:** Dùng `cnitool del` để thu hồi mạng sạch sẽ.

👉 Hãy tạm dừng video, mở terminal và bắt đầu gõ lệnh trong tệp `lab-guide.md` nhé!

PHẦN 2: Giải mã chiếc "hộp đen" CNI

Sau bài Lab, bạn đã phát hiện ra sự thật trần trụi về CNI:

- 🖱️ **CNI không phải là Daemon chạy ngầm!**

Nó không lắng nghe trên bất kỳ port nào, cũng không chạy liên tục. Nó đơn giản chỉ là các file thực thi (binary) nằm im lìm trên đĩa cứng của Node.

- 🖱️ **CNI là một cái HỢP ĐỒNG (Specification)!**

Hợp đồng này mô tả cách Container Runtime (như Kubelet) giao tiếp với CNI Plugins. Khi Pod được sinh ra, Kubelet chỉ cần **gọi trực tiếp** file binary đó, truyền dữ liệu JSON qua cổng vào tiêu chuẩn (`STDIN`) kèm một số biến môi trường, CNI Plugin làm xong việc và in trả lại kết quả cũng dưới dạng JSON!

Giao diện Giao tiếp chuẩn Đặc tả CNI

Khi Kubelet gọi CNI Binary (ví dụ: gọi file `/opt/cni/bin/bridge`), nó truyền thông tin qua 2 kênh:

1. Biến môi trường bắt buộc (Environment Variables)

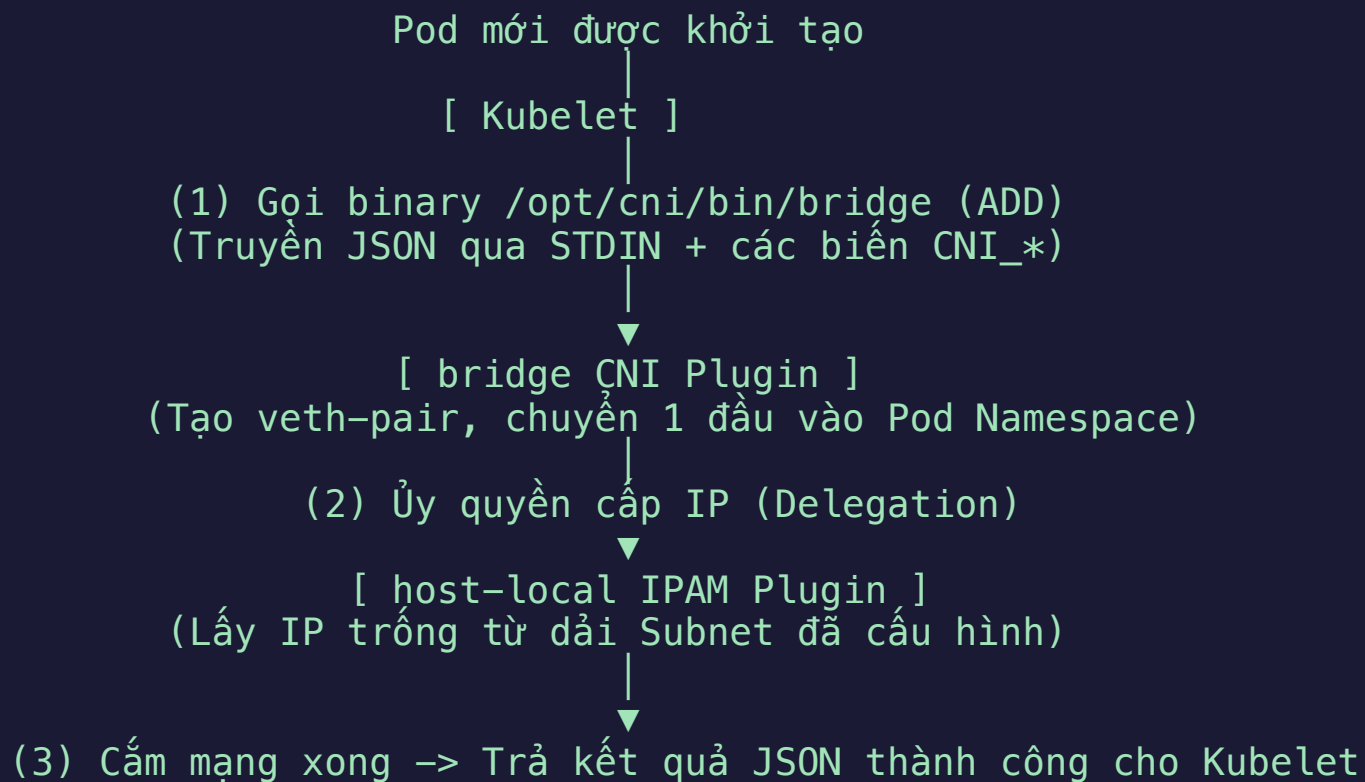
- `CNI_COMMAND` : Hành động yêu cầu (`ADD` , `DEL` , `CHECK` , hoặc `VERSION`).
- `CNI_CONTAINERID` : ID độc nhất của container.
- `CNI_NETNS` : Đường dẫn tới Network Namespace (`/var/run/netns/...`).
- `CNI_IFNAME` : Tên card mạng muốn đặt bên trong container (mặc định: `eth0`).
- `CNI_PATH` : Đường dẫn tới thư mục chứa các CNI binary (ví dụ: `/opt/cni/bin`). Kubelet (và `cnitool`) dùng biến này để tìm file thực thi cần gọi.

2. Cấu hình mạng mô tả qua JSON (`STDIN`)

- Truyền cấu hình dạng JSON (như file `.conflist` bạn vừa viết) mô tả loại card mạng (`bridge`), dải IP muốn cấp (`subnet`), gateway,... qua ngõ `STDIN`.

Kiến trúc tương tác: Kubelet ↔ CNI Plugins

Trong một cụm K8s thực tế, quy trình cắm mạng diễn ra tự động y hệt như những gì bạn vừa làm thủ công bằng `cnitool`:



Key Takeaways — Bài học cốt lõi

- **Đơn giản & Độc lập:** Đặc tả CNI được thiết kế cực kỳ tối giản. Bạn có thể sử dụng các CNI binaries này để tự xây dựng mạng cho các Linux container (Podman, containerd, hay netns thuần) mà không cần có Kubernetes.
- **Vòng đời ngắn (Stateless):** CNI plugins hoạt động theo kiểu "gọi xong rồi biến mất" (Stateless). Chúng chỉ khởi chạy, thực hiện cấu hình Linux (tạo card mạng, gán IP, route) trong vài mili-giây rồi tự chấm dứt tiến trình.
- **Hành trình tiếp theo:** Bây giờ bạn đã hiểu gốc rễ cách cắm mạng. Trong các tập tiếp theo, chúng ta sẽ bắt đầu mổ xẻ sâu kiến trúc của ba ông lớn CNI trong thế giới Production: **Flannel** → **Calico** → **Cilium**!

 **Chúc mừng bạn đã hoàn thành Phần 0 — Nền tảng K8s Networking!** Hãy sẵn sàng sang Phần 1 để khai phá CNI đầu tiên: *Flannel*!